JavaScript Basics – Detailed Explanation Guide

This guide explains every topic from your "Practical Sheet 06 – JavaScript Basics" in a simple and detailed way.

---

1. JavaScript Objects

A JavaScript object is a collection of related data and functions.

Example:

```
const student = {
    name: "John",
    age: 21,
    course: "IT"
};

console.log(student.name);
```

Output:

```
John
```

What is happening?

- `student` is an object.
- Inside the object there are properties:
    - `name`
    - `age`
    - `course`
- To access a property, use:

```
objectName.propertyName
```

Example:

```
student.age
```

Output:

```
21
```

Objects can also contain functions.

```
const student = {
    name: "John",
```

```
    greet: function() {
        return "Hello " + this.name;
    }
};

console.log(student.greet());
```

Output:

```
Hello John
```

`this.name` means “the name property inside this object”.

---

2. JavaScript Closures

A closure happens when a function remembers variables from outside it, even after the outer function has finished.

Example:

```
function outerFunction() {
    let count = 0;

    return function innerFunction() {
        count++;
        console.log(count);
    };
}

const counter = outerFunction();

counter();
counter();
counter();
```

Output:

```
1
2
3
```

Step-by-step explanation

Step 1

`outerFunction()` runs.

Inside it:

```
let count = 0;
```

Then it returns `innerFunction`.

Step 2

```
const counter = outerFunction();
```

Now `counter` stores the returned function.

Step 3

Every time `counter()` runs:

```
count++
```

The variable `count` still exists because the inner function remembers it.

That "remembering" ability is called a closure.

Closures are useful for:

- Counters
- Private variables
- Data hiding
- Timers

---

3. JSON Placeholder API

JSON Placeholder is a free fake API used for practice.

Example URL:

```
https://jsonplaceholder.typicode.com/users
```

Example code:

```
fetch('https://jsonplaceholder.typicode.com/users')
    .then(response => response.json())
    .then(data => console.log(data));
```

Explanation

`fetch()`

Sends a request to the API.

```
fetch(url)
```

`.then(response => response.json())`

The response comes as raw data.

`response.json()` converts it into JavaScript objects.

`.then(data => console.log(data))`

Prints the received data.

Example output:

```
[
  {
    id: 1,
    name: "Leanne Graham"
  }
]
```

You can also display only one value:

```
fetch('https://jsonplaceholder.typicode.com/users')
    .then(response => response.json())
    .then(data => {
        console.log(data[0].name);
    });
```

Output:

```
Leanne Graham
```

`data[0]` means the first user in the array.

ES6 New Features

ES6 is a newer version of JavaScript.

1a. Classes – Simple Constructor

A class is like a blueprint for creating objects.

```
class Student {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
}
```

```
const s1 = new Student("John", 20);

console.log(s1.name);
```

Output:

```
John
```

Explanation

- `class Student` creates a class.
- `constructor()` runs automatically when an object is created.
- `this.name = name` stores the given value inside the object.

```
new Student("John", 20)
```

creates an object.

1b. Class with Methods

```
class Student {
    constructor(name) {
        this.name = name;
    }

    greet() {
        console.log("Hello, my name is " + this.name);
    }
}

const student1 = new Student("Kasun");
student1.greet();
```

Output:

```
Hello, my name is Kasun
```

Explanation

`greet()` is a method inside the class.

When we do:

```
student1.greet();
```

The method runs.

KUPPI LAB

1c. Class Inheritance

Inheritance means one class can use features from another class.

```
class Animal {
    speak() {
        console.log("Animal makes a sound");
    }
}

class Dog extends Animal {
    bark() {
        console.log("Dog barks");
    }
}

const myDog = new Dog();

myDog.bark();
myDog.speak();
```

Output:

```
Dog barks
Animal makes a sound
```

Explanation

```
class Dog extends Animal
```

means `Dog` inherits from `Animal`.

So `myDog` can use:

- `bark()` from `Dog`
- `speak()` from `Animal`

---

2. Variables – var, let, const

---

var

```
var x = 10;
var x = 20;

console.log(x);
```

Output:

```
20
```

`var` can:

- be changed
- be declared again

Problem: It can create bugs.

---

let

```
let y = 10;
y = 30;

console.log(y);
```

Output:

```
30
```

But this is NOT allowed:

```
let y = 10;
let y = 20;
```

Because `let` cannot be declared again in the same block.

---

const

```
const z = 50;
```

You cannot change it.

```
z = 100;
```

This gives an error.

Use `const` when the value should stay the same.

Scope Difference

```
if (true) {
    var a = 1;
    let b = 2;
}
```

KUPPI LAB

```
console.log(a); // works
console.log(b); // error
```

Why?

- `var` ignores the block.
- `let` stays only inside the block.

---

3. Array Methods – map()

`map()` goes through each item in an array and creates a new array.

```
const numbers = [1, 2, 3, 4];

const doubled = numbers.map(num => num * 2);

console.log(doubled);
```

Output:

```
[2, 4, 6, 8]
```

Explanation

For each number:

- 1 → 2
- 2 → 4
- 3 → 6
- 4 → 8

The original array does not change.

```
console.log(numbers);
```

Still:

```
[1, 2, 3, 4]
```

Example with names:

```
const users = ["John", "Anne", "Mike"];

const list = users.map(user => "Hello " + user);

console.log(list);
```

Output:

```
[
  "Hello John",
  "Hello Anne",
  "Hello Mike"
]
```

4a. Destructuring Arrays

Destructuring means taking values from an array or object and storing them in variables.

```
function getValues() {
    return [10, 20];
}

const [a, b] = getValues();

console.log(a);
console.log(b);
```

Output:

```
10
20
```

Explanation

The function returns:

```
[10, 20]
```

Then:

```
const [a, b]
```

means:

- `a = 10`
- `b = 20`

Without destructuring, you would write:

```
const values = getValues();
const a = values[0];
const b = values[1];
```

Destructuring is shorter and cleaner.

4b. Deeply Nested Object Destructuring

Example:

```
const person = {
    name: "Nimal",
    address: {
        city: "Colombo",
        country: "Sri Lanka"
    }
};

const {
    address: { city, country }
} = person;

console.log(city);
console.log(country);
```

Output:

```
Colombo
Sri Lanka
```

Explanation

Inside `person`, there is another object called `address`.

```
address: {
    city: "Colombo",
    country: "Sri Lanka"
}
```

Destructuring goes inside that object and directly gets:

- `city`
- `country`

Self-Study – Asynchronous JavaScript

Asynchronous JavaScript means JavaScript can do something that takes time (like loading data from the internet) without stopping the whole program.

---

1. Callback Function

```
function fetchData(url, callback) {
    const xhr = new XMLHttpRequest();

    xhr.open("GET", url);
```

```
    xhr.onload = function () {
        if (xhr.status === 200) {
            callback(null, xhr.responseText);
        } else {
            callback("Error occurred", null);
        }
    };

    xhr.send();
}

fetchData("https://jsonplaceholder.typicode.com/posts/1", function(error,
data) {
    if (error) {
        console.log(error);
    } else {
        console.log(data);
    }
});
```

Explanation

The callback function is passed into `fetchData()`.

Later, when the data arrives, the callback is executed.

```
callback(null, xhr.responseText)
```

means:

- no error
- send the data

```
callback("Error occurred", null)
```

means:

- there is an error

---

2. Promises

Promises are a cleaner way to handle asynchronous work.

```
function fetchData(url) {
    return new Promise((resolve, reject) => {
        const xhr = new XMLHttpRequest();

        xhr.open("GET", url);

        xhr.onload = function () {
            if (xhr.status === 200) {
                resolve(xhr.responseText);
```

```
            } else {
                reject("Error occurred");
            }
        };

        xhr.send();
    });
}

fetchData("https://jsonplaceholder.typicode.com/posts/1")
    .then(data => {
        console.log(data);
    })
    .catch(error => {
        console.log(error);
    });
```

Explanation

- `resolve()` means success.
- `reject()` means failure.
- `.then()` runs if success happens.
- `.catch()` runs if error happens.

---

3. async and await

`async` and `await` make asynchronous code look like normal code.

```
function fetchData(url) {
    return new Promise((resolve, reject) => {
        const xhr = new XMLHttpRequest();

        xhr.open("GET", url);

        xhr.onload = function () {
            if (xhr.status === 200) {
                resolve(xhr.responseText);
            } else {
                reject("Error occurred");
            }
        };

        xhr.send();
    });
}

async function fetchDataAsync() {
    try {
        const data = await
fetchData("https://jsonplaceholder.typicode.com/posts/1");
        console.log(data);
    } catch (error) {
        console.log(error);
    }
```

```
}

fetchDataAsync();
```

Explanation

`async`

Makes a function asynchronous.

`await`

Waits until the Promise finishes.

Instead of:

```
.then(...)
```

we can write:

```
const data = await fetchData(...)
```

This is easier to read.

Final Quick Summary

Topic	Main Idea
Object	Stores related data
Closure	Function remembers outside variables
JSON Placeholder	Fake API for testing
Class	Blueprint for objects
Inheritance	One class uses another class
var	Old variable type
let	Variable that can change
const	Variable that cannot change
map()	Creates a new array
Destructuring	Takes values from arrays/objects
Callback	Function passed into another function
Promise	Handles future result
async/await	Cleaner way to use promises

KUPPI LAB